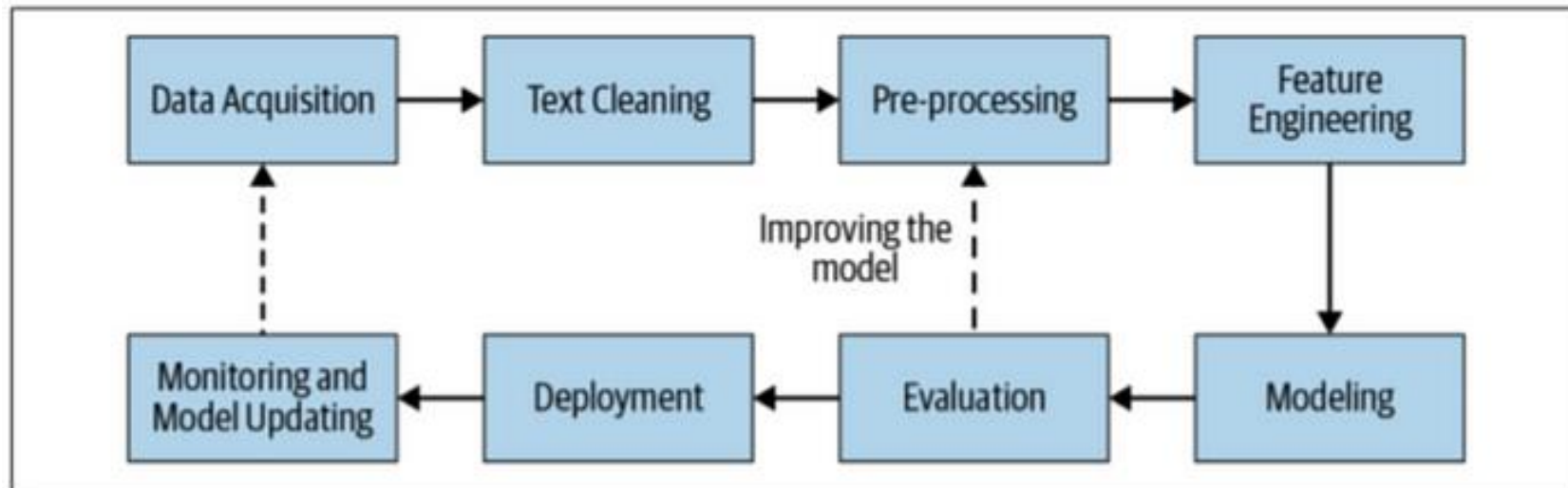


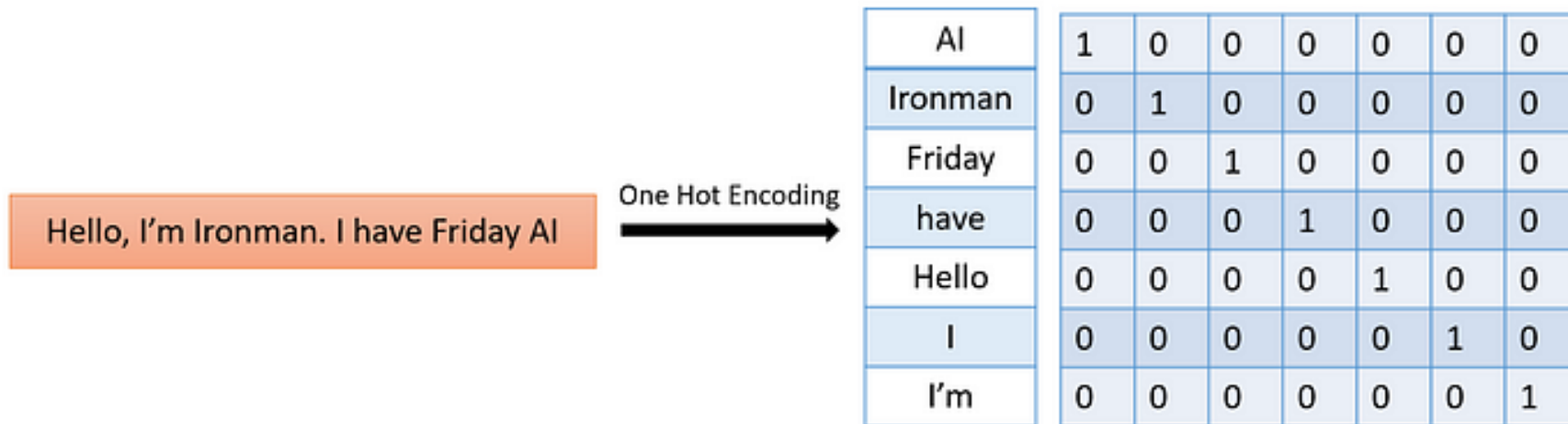
Feature Engineering

- Feature engineering in Natural Language Processing (NLP) involves **transforming raw text** data into **numerical features** that machine learning models can comprehend and utilize effectively.
- Natural language is inherently **ambiguous**, with words often having **multiple meanings** depending on the **context**.
- The goal is to **represent text** in a format that captures **semantic meaning**, **contextual** information, and **relationships** between **words**.



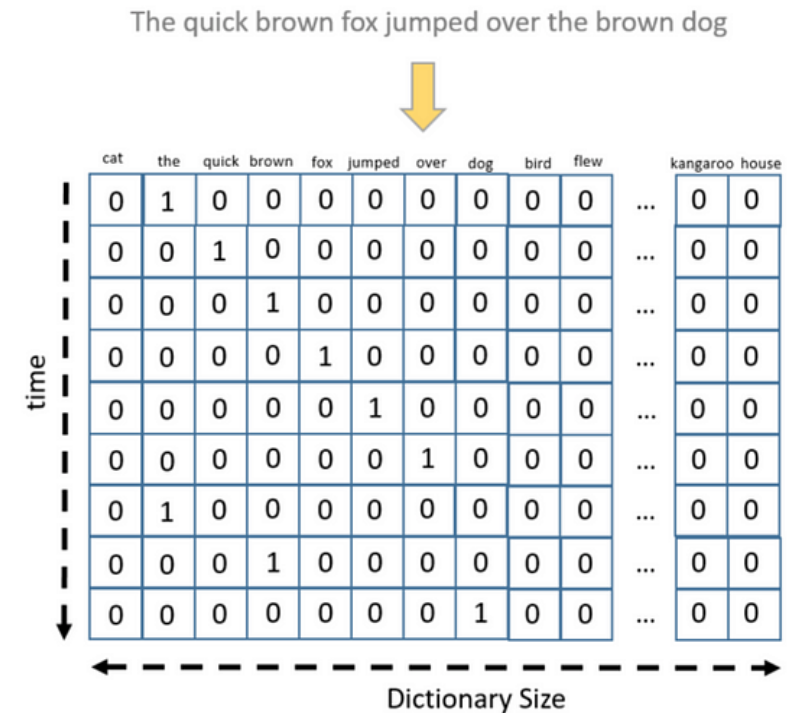
Text Representation: One-Hot Encoding

- Represents words as **binary vectors**, where each word has a unique index in the vector.
- Converts words into a **high-dimensional space**, with each dimension corresponding to a unique word.



Text Representation: One-Hot Encoding

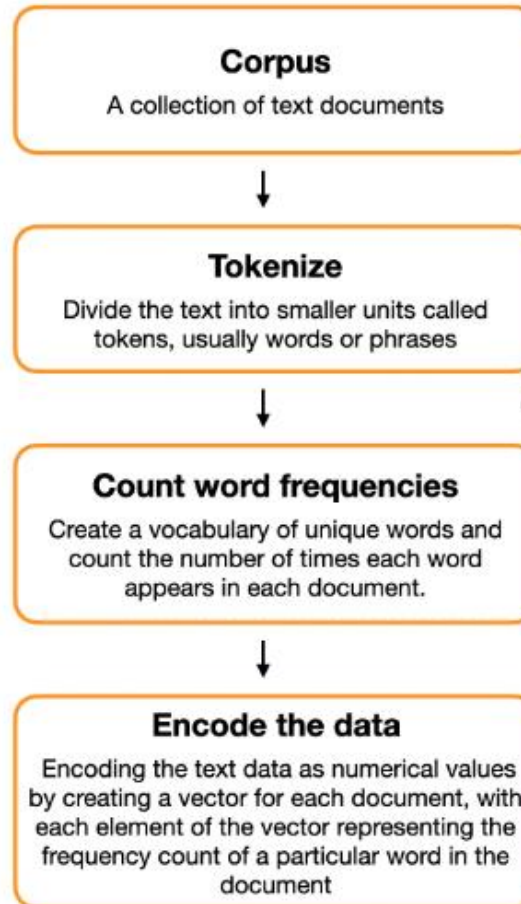
- **Advantages:**
 - **Simple Representation:** Each category or word is represented by a binary vector, making it **easy** to **understand** and **implement**.
- **Disadvantages:**
 - **Dimensionality:** It can lead to a **high-dimensional feature space**, especially when dealing with large categorical variables
 - **Sparse Representation:** It produces **sparse matrices** with mostly zeros, which can lead to **memory inefficiency** and **computational overhead**.



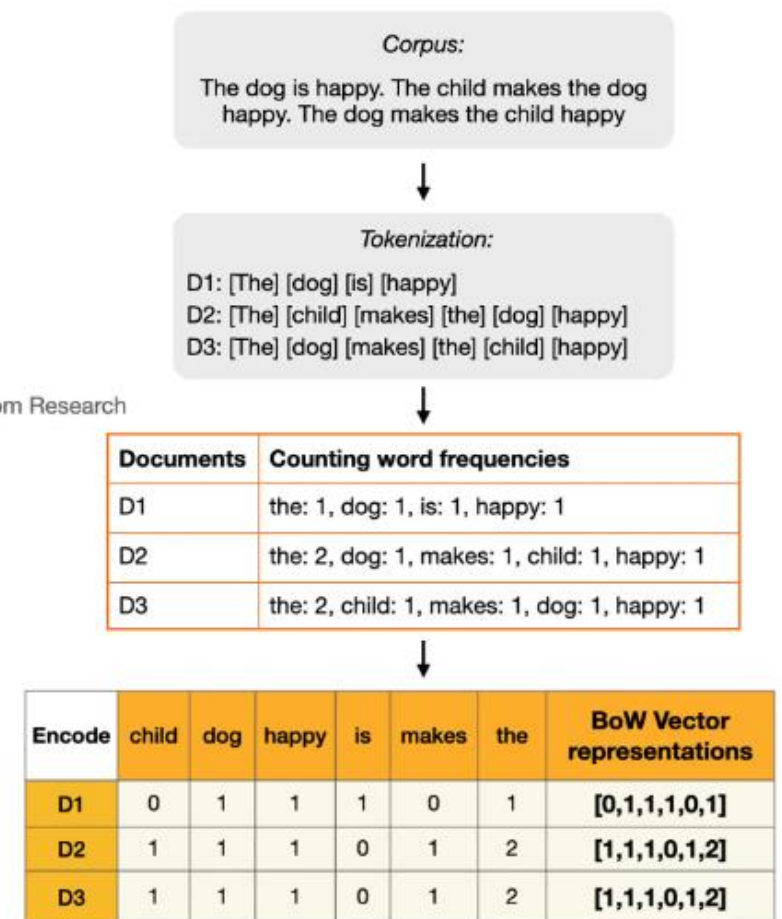
Text Representation: Bag of Words (BoW)

- Introduced in the 1950s and 1960s, the Bag of Words model is one of the earliest and **simplest** approaches for representing text.
- Represents text as a collection of unique words **disregarding grammar** or **word order**.
- Creates a **matrix** where rows represent **documents** and **columns** represent **unique words**, with values indicating word **occurrence** frequencies.
- Unlike one-hot encoding, BoW captures **word frequency** rather than binary presence.

Bag of Words Model explanation



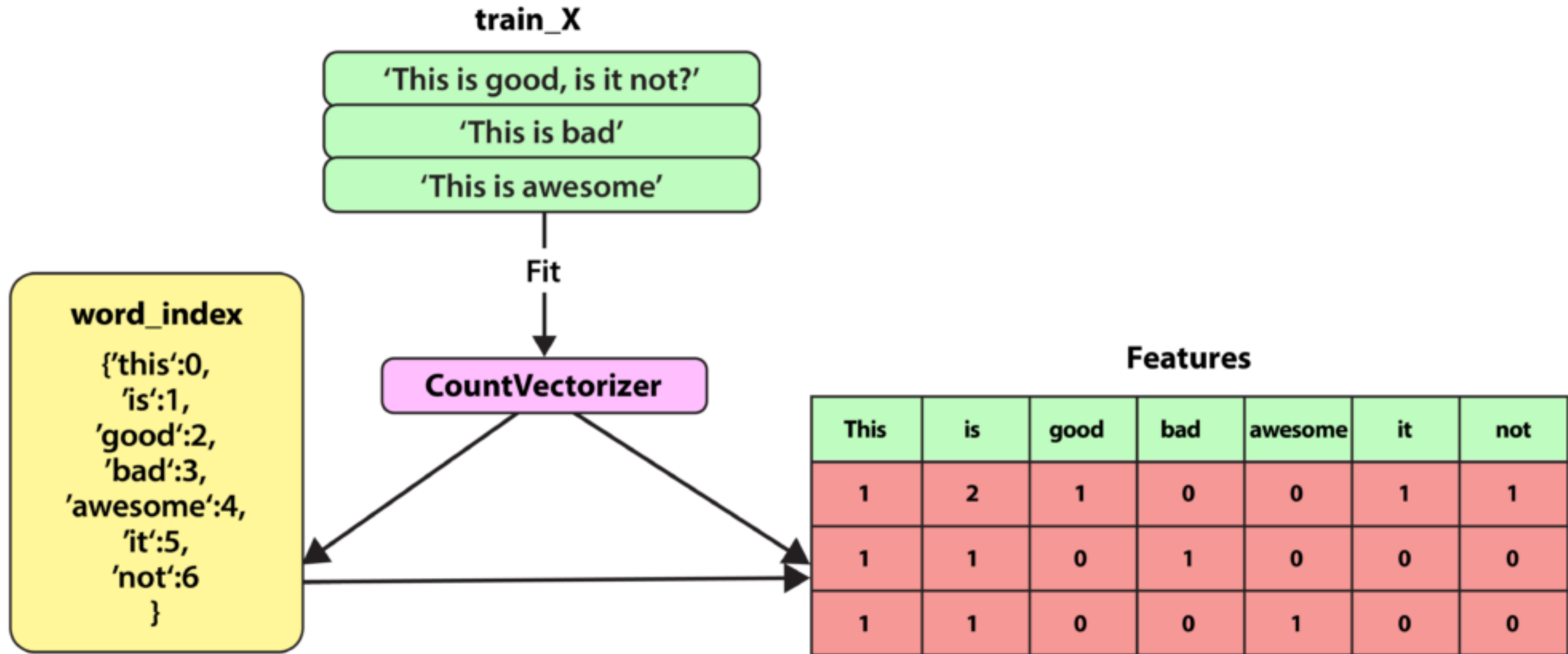
Example



Title: Bag of Words model explained using an example

Source: AIML.com Research

TOKENIZERS: BAG-OF-WORDS



Bag-of-Words (through the CountVectorizer method) encodes the total number of times a document uses each word in the associated corpus.

BOW Advantages

- **Simplicity:** BoW is a relatively simple and **straightforward** approach.
- **Efficiency:** It's **computationally efficient**, making it suitable for large datasets.
- **Interpretability:** The results are often **interpretable**, as they are based on word frequencies.

BOW Disadvantages

- **Loss of Semantic Information:** BoW treats words as **individual units without** considering their **order** or **context**. This can lead to the **loss** of important **semantic information**.
- **High Dimensionality:** BoW representations can become very **high-dimensional**, especially for **large vocabularies**. This can lead to **computational** challenges and potential **overfitting**.
- **Sensitivity to Stop Words:** Stop words (common words like "the," "and," "a") often have **little semantic value**. However, BoW models may assign **significant weight** to these words, which can **affect** the **accuracy** of the model.
- **Inability to Capture Synonymy and Polysemy** (*words with multiple meanings*).

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Two documents with different meanings, yet same BoW representation
(Source: AIML.com Research)



© AIML.com Research

The baseball **pitcher** drank a **pitcher** of water

Word 'Pitcher' has two different meanings, yet is represented by same feature
(Source: AIML.com Research)

Text Representation: TF-IDF

Term Frequency – Inverse Document Frequency.

- Measures the **importance** of **words** in a document **relative** to a **corpus**.
- Considers both the **frequency** of a term in a **document** and its **rarity across** the **corpus**.
- Assigns **higher weights** to **rare terms** that are more discriminative.

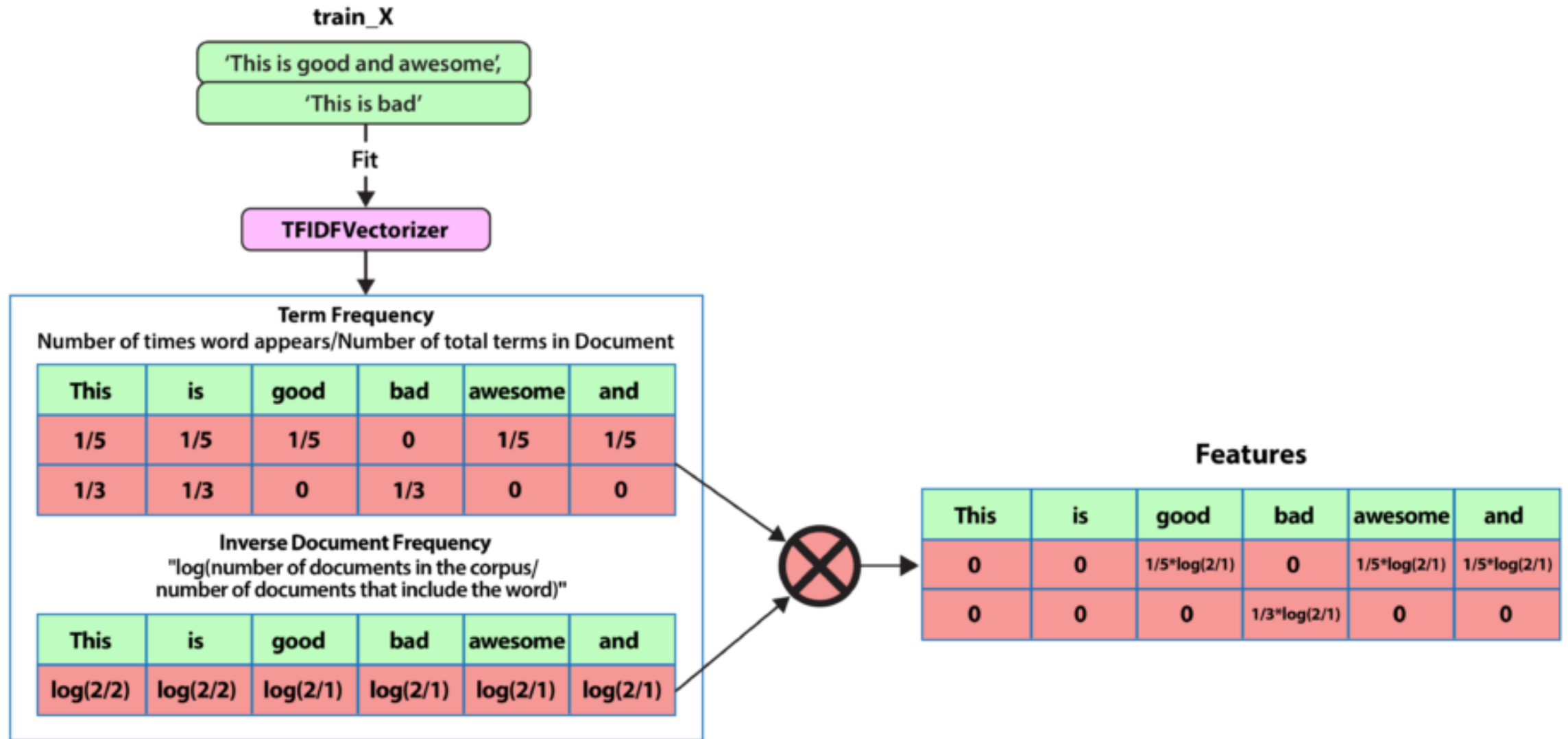
$$tfidf_{t,d} = tf_{t,d} \cdot idf_t$$

$tf_{t,d}$ = term frequency of t in d

idf_t = inverse document frequency of t



TOKENIZERS: TERM FREQUENCY - INVERSE DOCUMENT FREQUENCY (TF-IDF)



TF-IDF creates features for each document based on how often each word shows up in a document versus the entire corpus.

Text Representation: TF-IDF (Advantage)

- By combining TF and IDF, TF-IDF effectively **balances** the **importance** of **frequent terms** within a **document** and **rare terms across** the **corpus**, improving the representation of important terms.
- **Common words** (e.g., "the", "and") that appear frequently across many documents are **down-weighted**, helping to **focus** on **more meaningful terms**.
- TF-IDF enhances the relevance of documents based on their **unique terms**. This makes it useful for tasks such as **search engine** ranking and **document retrieval**, where finding documents with unique, relevant terms is crucial.
- The TF-IDF method is relatively **straightforward** to **compute** and **interpret**, making it accessible and effective for various text processing tasks.
- By emphasizing the importance of unique terms, TF-IDF can **improve** the **accuracy** of **document similarity** measures, such as **cosine similarity**.

Text Representation: TF-IDF (Disadvantage)

- TF-IDF represents text as a bag of words, **ignoring** the **order** and **context** of words. This can lead to loss of **syntactic** and **semantic** information.
- The representation can lead to very **high-dimensional vectors**, especially with large vocabularies. This can result in increased computational **costs** and **memory** usage.
- TF-IDF vectors are often **sparse**, with many zero values. Sparse representations can be less efficient for certain algorithms and may require specialized data structures and techniques.
- While TF-IDF can highlight important terms, it may still **struggle** with **extremely rare** or **domain-specific terms** that do not appear frequently enough **across documents** to achieve meaningful IDF scores.
- TF-IDF does not address **polysemy**, where a word may have **multiple meanings** depending on context. All **instances** of a **term** are **treated uniformly**, regardless of different meanings.

N-Gram

- N-grams capture **local context** and word order within a **fixed window**. This can be useful for understanding phrases and patterns that are specific to a document or corpus.
- The concept of n-grams is straightforward and easy to implement.
- N-grams are effective for feature extraction in **text classification** and **clustering**. They help represent text in a way that **captures specific word combinations** that might be relevant for the task.

Text: "In consciousness studies the consciousness studies are fundamental"

Unigram

In	consciousness	studies	the	are	fundamental
----	---------------	---------	-----	-----	-------------

Bigram

In consciousness	consciousness studies	studies the	the consciousness	studies are	are fundamental
------------------	-----------------------	-------------	-------------------	-------------	-----------------

Trigram

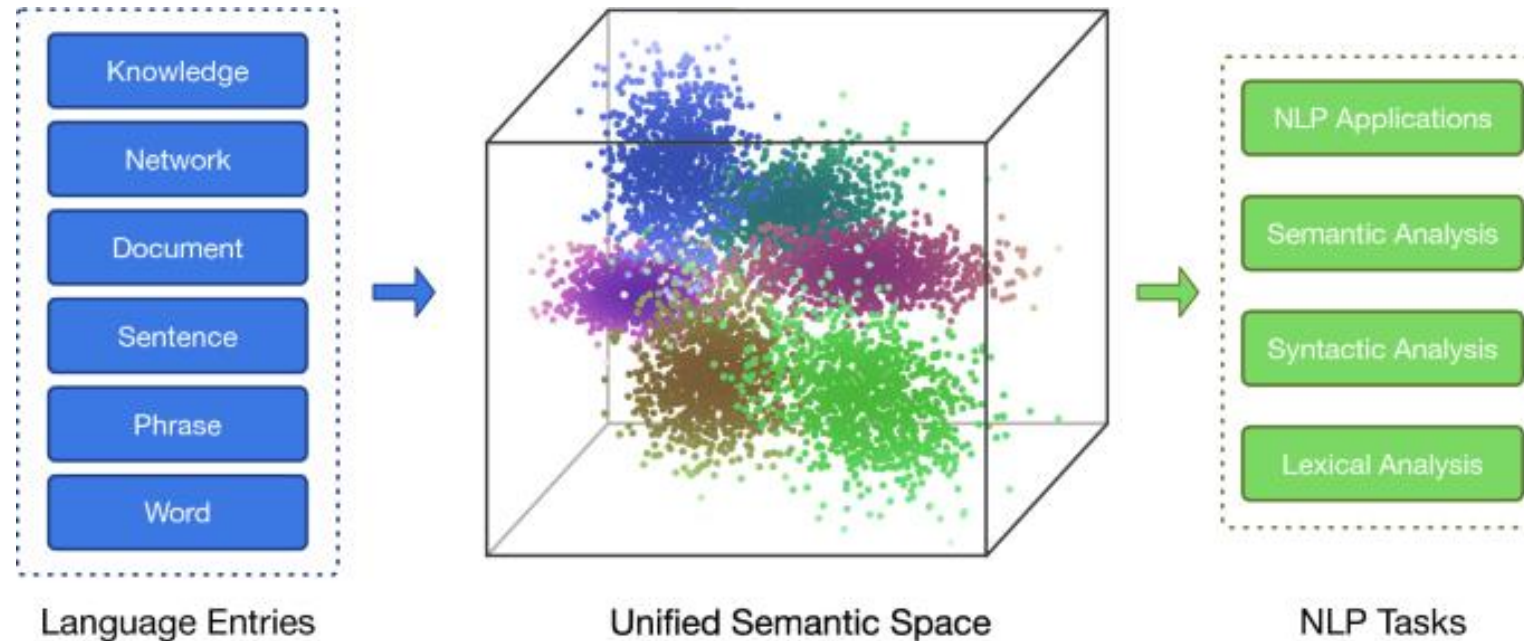
In consciousness studies	consciousness studies the	studies the consciousness	the consciousness studies	consciousness studies are	studies are fundamental
--------------------------	---------------------------	---------------------------	---------------------------	---------------------------	-------------------------

Neural language models

- Word embedding can help solve the sparsity problem since we can ditch the one-hot representation of words for their vectors.
- Therefore we can easily build **neural language models** that can **take into consideration more words** than the **n-gram models**.
- Despite this advantage, however, the **number of words** considered **remains fixed**, and so neural language models still **struggle** with **long-term dependencies**.

Unified Semantic Space

- A unified semantic space is a **high-dimensional vector space** where **words, phrases, sentences**, or other **linguistic units** are **represented** as **vectors**.
- These **vectors capture** the **meaning** of the **linguistic units** in a way that allows for **meaningful comparisons** and **manipulations**.



Popular Word Embedding Techniques

- Traditional Word Embedding Techniques
- Advanced Word Embedding Techniques
- Contextualized Word Embedding Techniques

Traditional Word Embedding Techniques

- **One-hot encoding:**

- One-hot encoding is a simple technique for representing words as **binary vectors**. Each word in the vocabulary is assigned a unique position in the vector, with a value of 1 at that position and 0 elsewhere. While this method is straightforward, it suffers from **high dimensionality** and **fails** to capture any **semantic** relationships between words.

- **TF-IDF (Term Frequency-Inverse Document Frequency):**

- TF-IDF is a popular technique for representing text data in a **numerical** format. It combines the **frequency** of a word in a document (Term Frequency) with the inverse of its frequency across all documents (Inverse Document Frequency). This method weighs the importance of words, but it **does not capture** the **context** or **semantic** relationships between words.

- **Bag of words:**

- The **bag of words** approach represents documents as a **vector** of **word frequencies**. Although this method can **capture** the **importance** of **words** within a document, it **ignores** the **order** of words and **fails** to encode any **semantic** information.

Neural Word Embedding Techniques

- **Word2Vec:**

- Word2Vec is a popular neural network-based word embedding technique that **captures semantic relationships** between **words**. It comes in two flavors:
 - Continuous Bag of Words (CBOW)
 - Skip-Gram.
- **CBOW** predicts the **target word** based on its **context**, while **Skip-Gram** predicts the **context words** given the **target** word. Word2Vec efficiently learns high-quality vector representations, **capturing semantic** and syntactic information.

- **GloVe (Global Vectors for Word Representation):**

- GloVe constructs a **word-word co-occurrence matrix** and learns the **embeddings** by **minimizing** the **difference** between the **dot product** of word vectors and the logarithm of their co-occurrence count.

- **FastText:**

- FastText is an **extension** of the **Word2Vec** model, which represents words as the sum of their **subword** (n-gram) vectors. This technique allows FastText to capture **morphological** information and generate embeddings for **out-of-vocabulary words**.

Contextualized Word Embedding Techniques

Unlike Word2Vec, GloVe, and FastText, which **assign** a **single vector** to a **word regardless** of its **context**, contextualized embeddings **generate different embeddings** for a **word depending** on its **context**. These embeddings are more dynamic and capture meaning based on **sentence-level context**.

ELMo (Embeddings from Language Models):

Uses a **bidirectional LSTM** trained on a language modeling task to generate word embeddings. ELMo generates **context-dependent** embeddings by considering the **entire sentence** to determine the representation of each word.

BERT (Bidirectional Encoder Representations from Transformers):

A **transformer-based model** that learns context-aware word embeddings by processing the **entire sentence from both the left and the right**. Unlike ELMo, which uses LSTMs, BERT uses **self-attention mechanisms** and is **bidirectional**, capturing **richer context**.

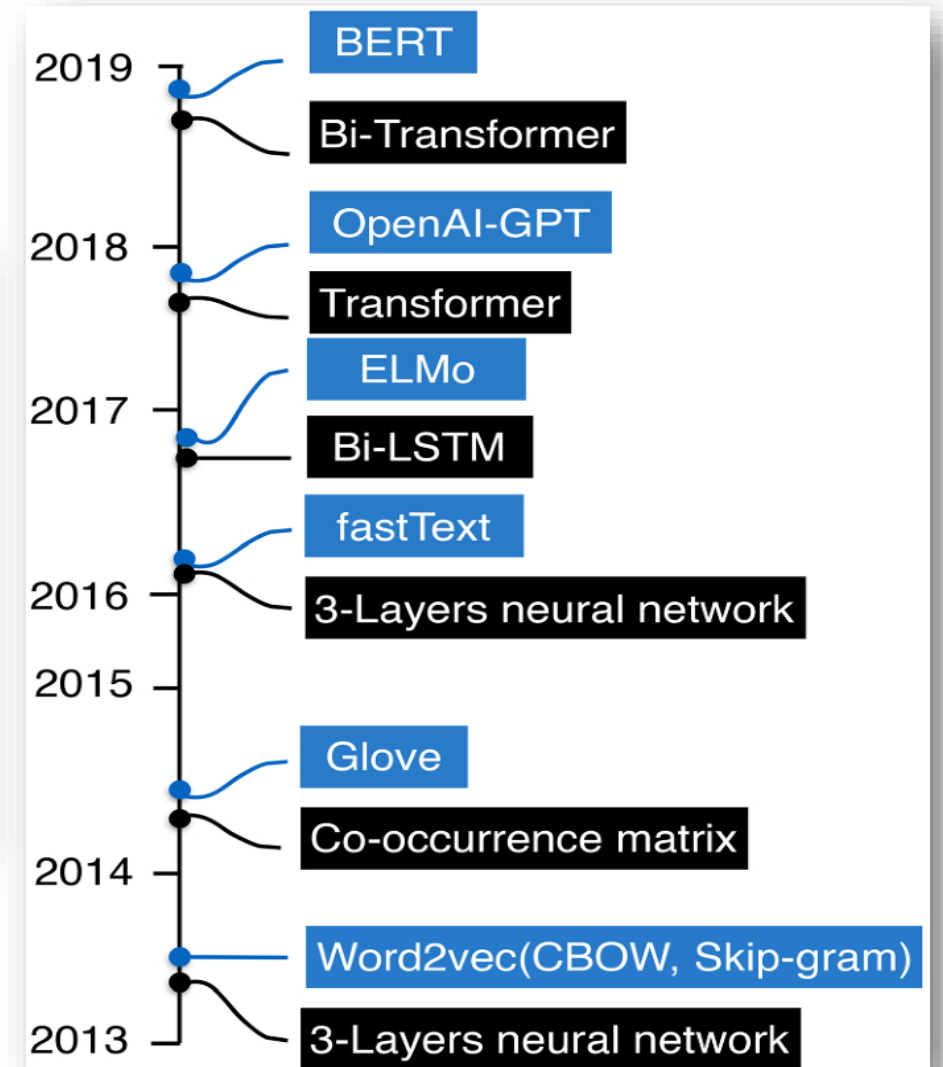
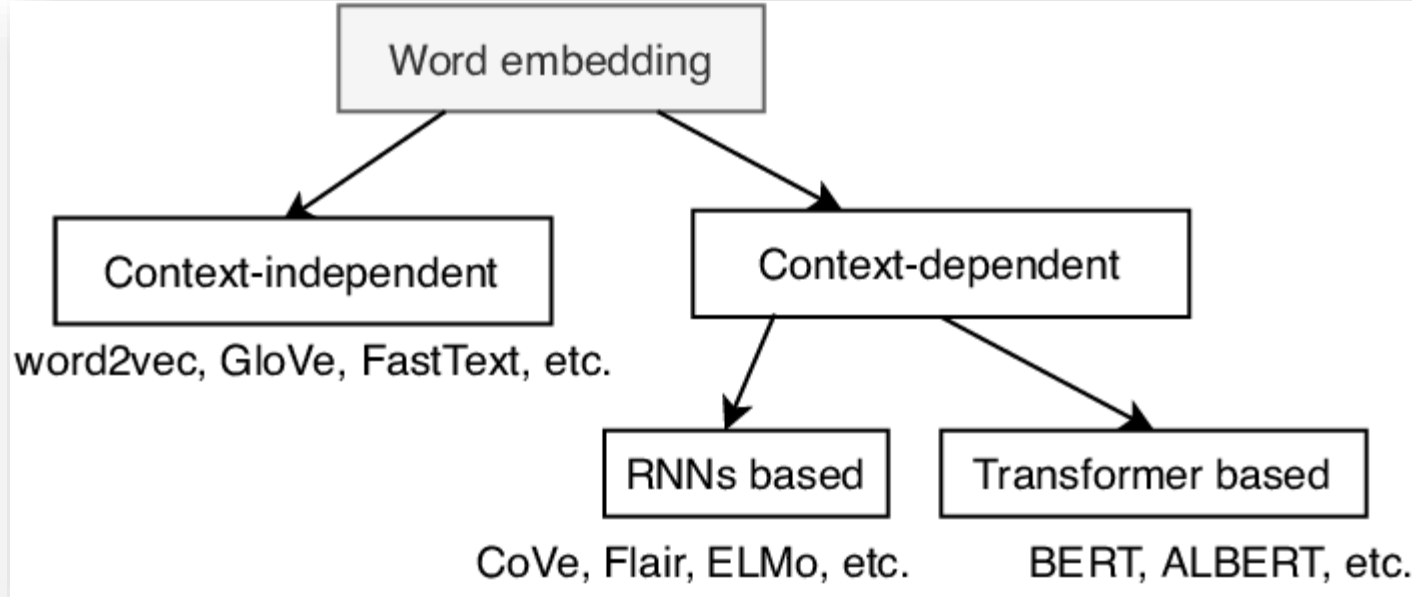
GPT (Generative Pre-trained Transformer):

GPT uses a **unidirectional transformer** to generate context-aware embeddings. It is trained on a large corpus using a language modeling objective. However, it differs from BERT as it predicts context from **left to right**.

T5 (Text-to-Text Transfer Transformer):

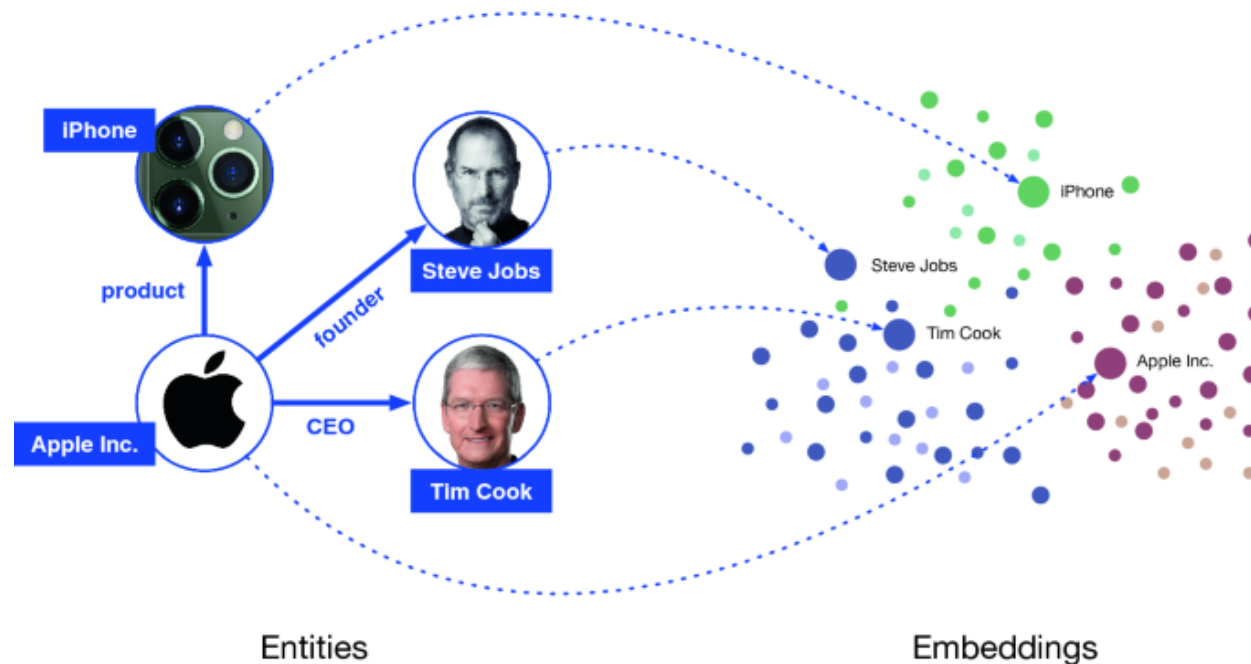
T5 is another **transformer-based model** that learns embeddings in a text-to-text framework, where every task (e.g., classification, translation) is cast into a text generation task. Like BERT, T5 embeddings are context-sensitive.

Word Embedding



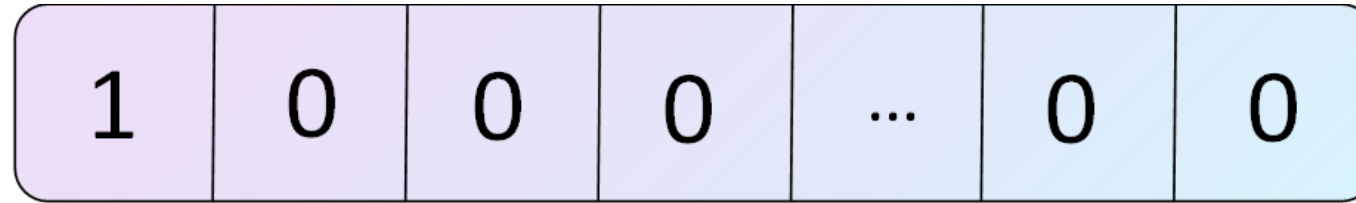
Word Embeddings (Word2Vec, GloVe, FastText)

- Word embeddings are a type of **text representation** that map **words** or **phrases** into **continuous vector spaces**.
- These vectors **capture semantic meanings** and **relationships** between **words**, allowing for more nuanced text analysis compared to traditional methods like **Bag of Words** or **TF-IDF**.

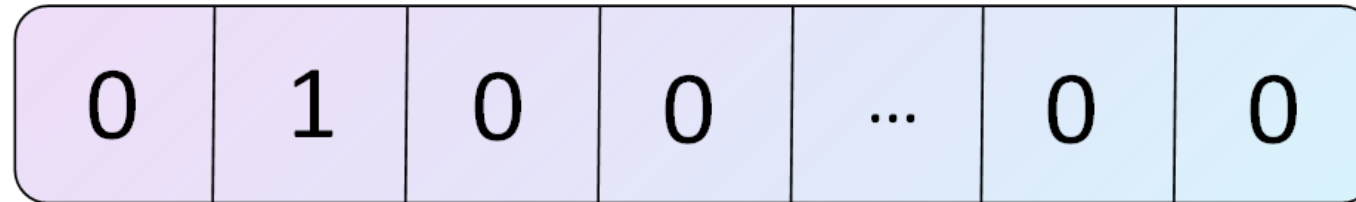


Vector space models capture **semantic meaning** and **relationships** between **words**.

one-hot encoding for words.



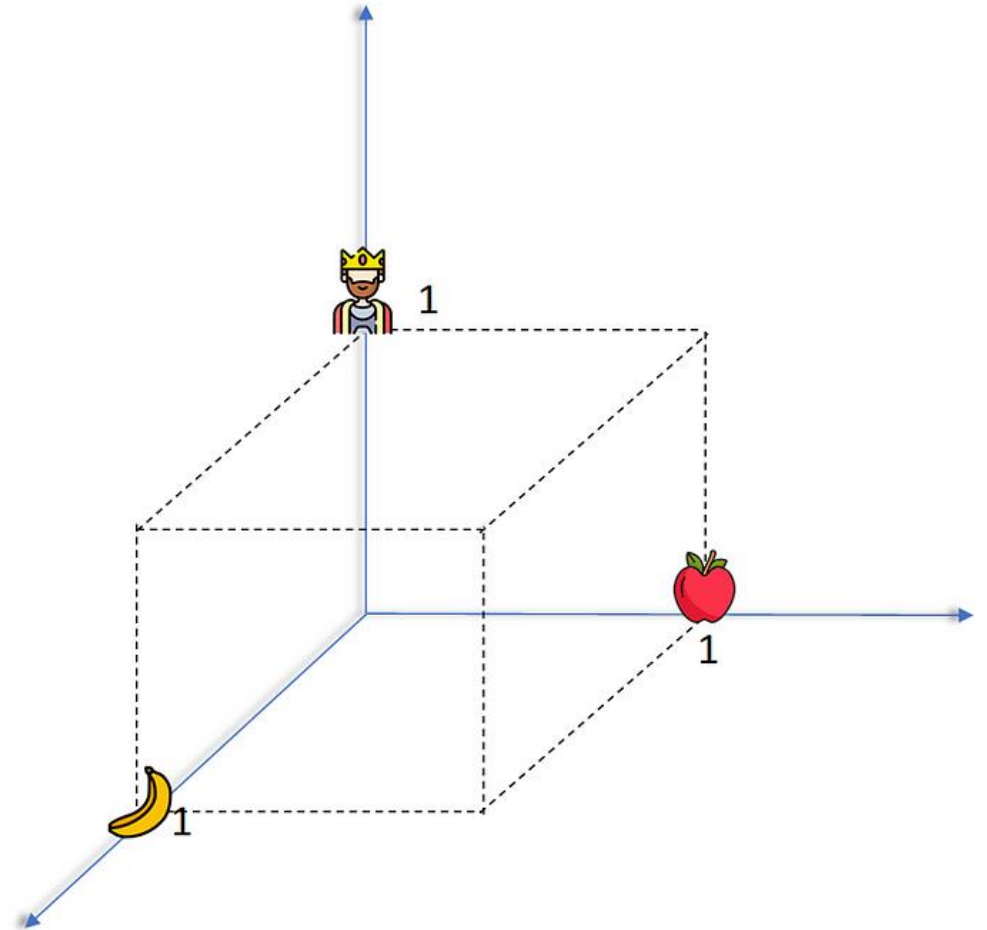
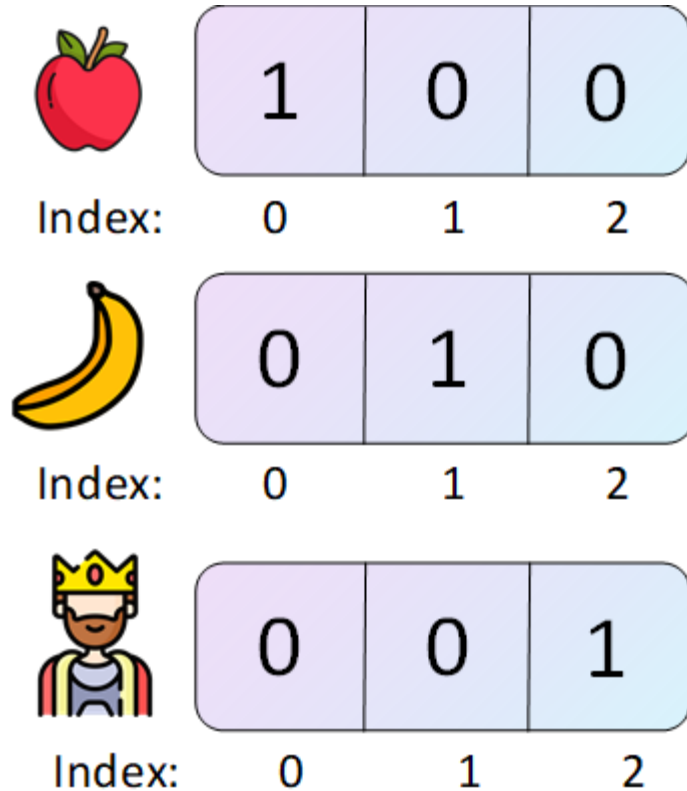
Index: 0 1 2 3 ... 99998 99999



Index: 0 1 2 3 ... 99998 99999

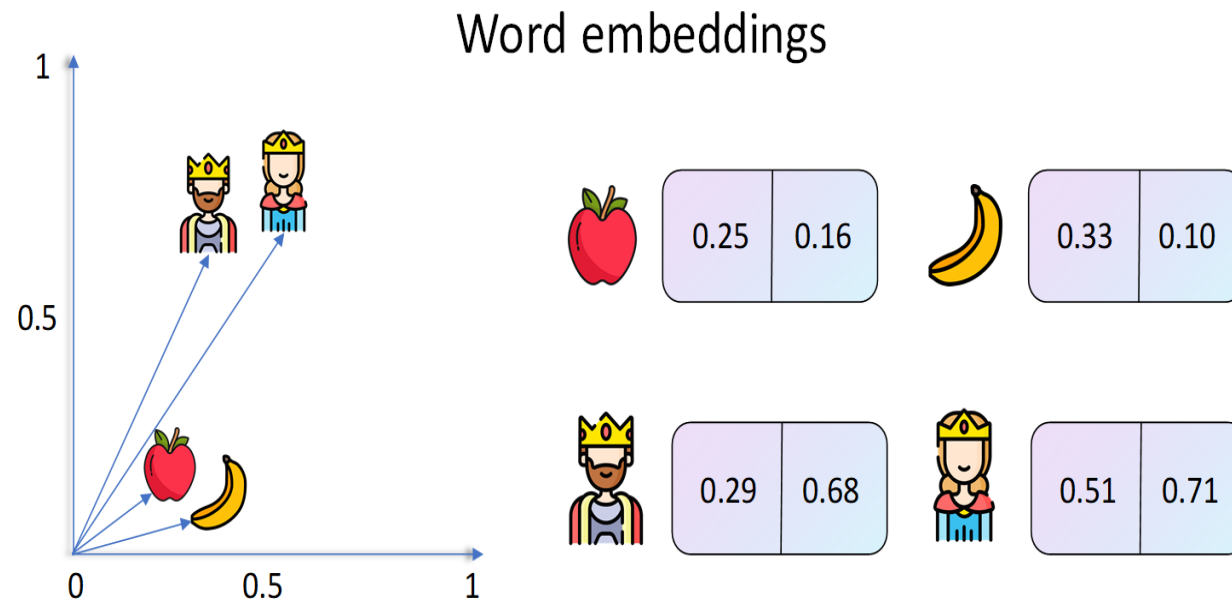
one-hot encoding for words.

One hot vector representations of our vocabulary

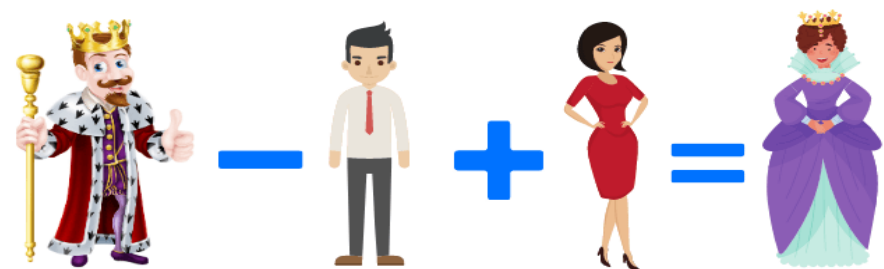
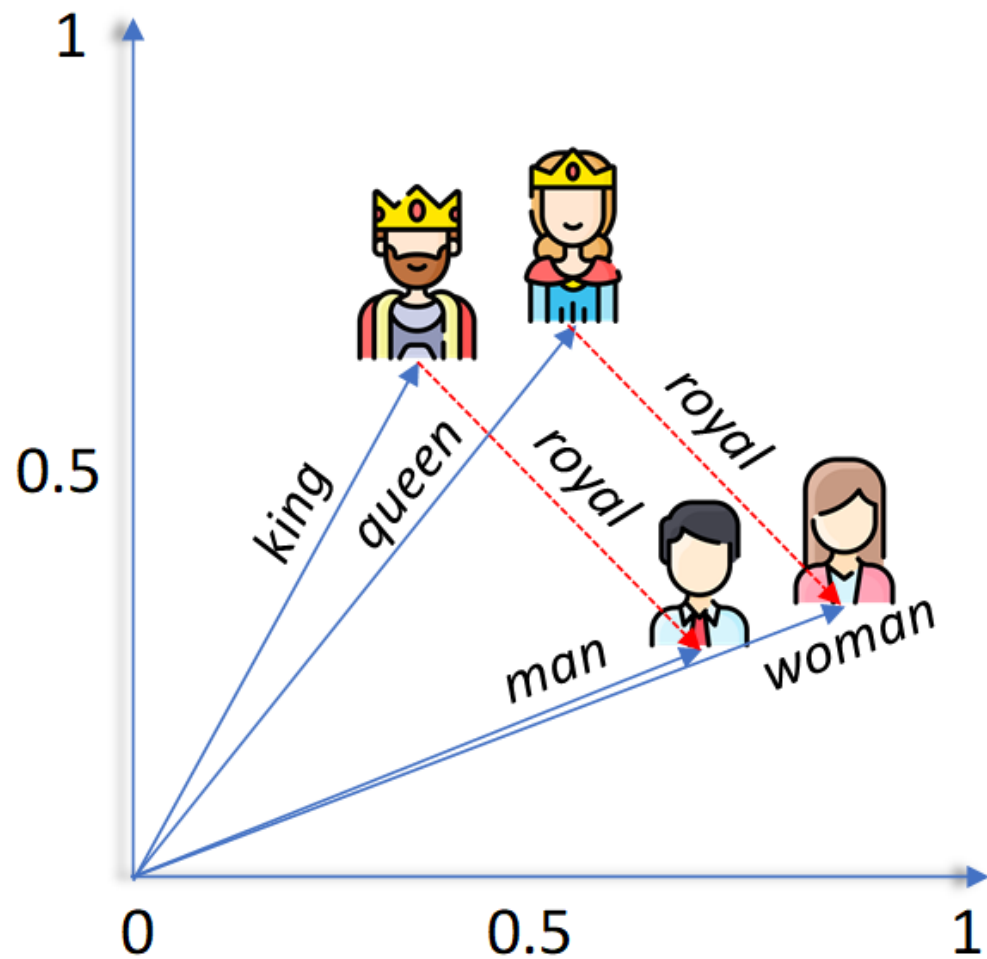


Word embeddings

- **Word embeddings** solve these problems by **representing** each **word** in the vocabulary by a fairly small (150, 300, 500 dimensional) **fixed size vector**, called an embedding, which is **learned** during the **training**.
- These vectors are created in a manner so that **words that appear in similar contexts** or have similar meaning **are close together**, and they are **not sparse** vectors like the ones derived from one-hot embeddings.



Word embeddings



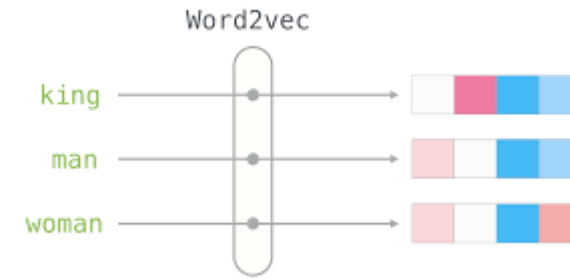
dataaspirant.com

How are word embeddings built?

Learning from Data

- Consider the sentence “*I love drinking apple smoothies*”. If I **remove** the word ‘*apple*’ we are left with the following, incomplete sentence: ‘*I love drinking __ smoothies*’. If I then gave you this incomplete sentence, and told you to guess the missing word, you will probably say words like ‘*banana*’, ‘*strawberry*’, or ‘*apple*’, which all have a similar meaning, and usually appear in similar contexts.
- One of the main ways to learn word embeddings, is by a **very similar process to this**: the algorithms learn similar word embedding for words that appear many times in similar contexts by **guessing missing words** in a huge corpus of text sentences.
- An **embedding matrix E** (the matrix that translates a one hot embedding into a word embedding vector) **is calculated** by training something similar to a language model (a model that tries to predicts missing words in a sentence) **using an Artificial Neural Network to predict this missing word**, in a similar manner to how the weights and biases of the network are calculated.

Word2Vec



- **Word2Vec** is a word embedding model developed by **Google** that learns vector representations of words based on their context in a corpus.
- word2vec is **not** a **singular algorithm**, rather, it is a **family** of model architectures and optimizations that can be used to **learn word embeddings** from **large datasets**.
- Embeddings learned through word2vec have proven to be successful on a variety of downstream natural language processing tasks.
- **Continuous bag-of-words model**: **predicts** the **middle word** based on **surrounding** context **words**. The context consists of a few words before and after the current (middle) word. This architecture is called a bag-of-words model as the order of words in the context is not important.
- **Continuous skip-gram model**: **predicts words** within a certain range **before** and **after** the **current word** in the same sentence.

CBOW

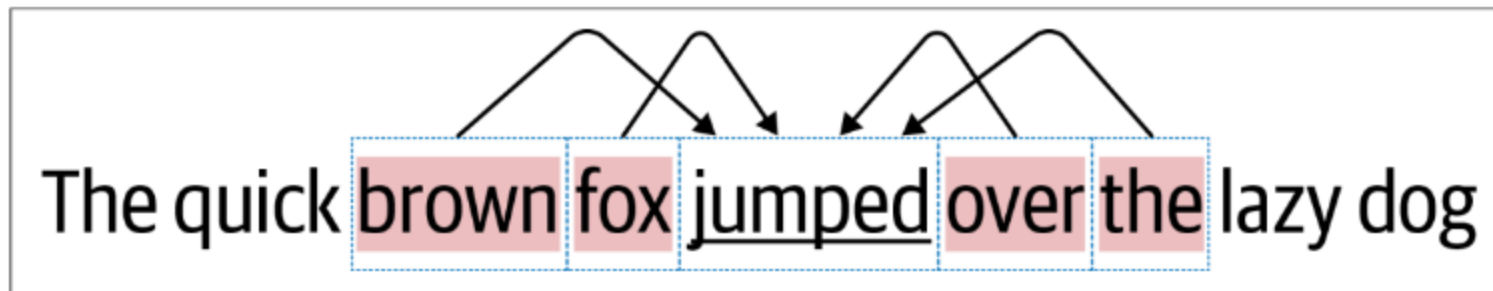


Figure 3-7. CBOW: given the context words, predict the center word

Source Text	Training Samples (context, target)
The quick brown fox jumps over the lazy dog.	→ ((quick, brown), The)
The quick brown fox jumps over the lazy dog.	→ ((The, brown, fox), quick)
The quick brown fox jumps over the lazy dog.	→ ((The, quick, fox, jumps), brown)
The quick brown fox jumps over the lazy dog.	→ ((quick, brown, jumps, over), fox)

Figure 3-8. Preparing a dataset for CBOW

SkipGram

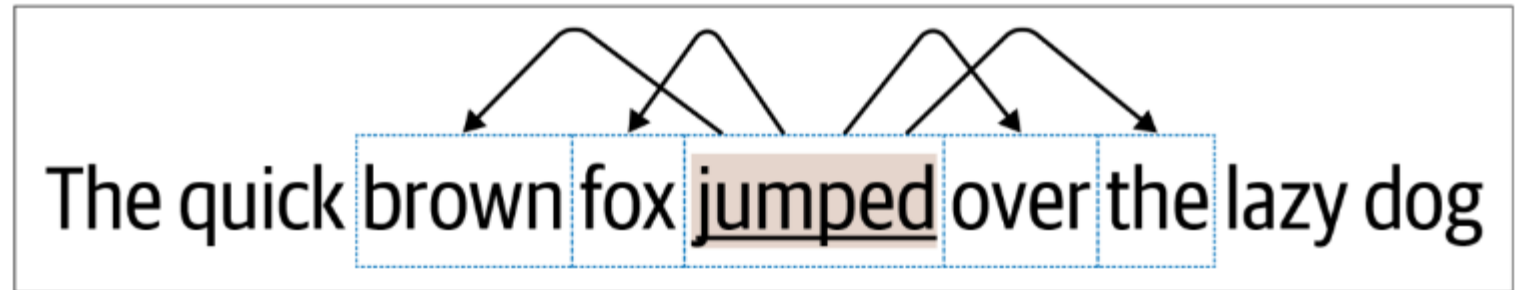


Figure 3-10. SkipGram: given the center word, predict every word in context

Source Text	Training Samples (target, context)
The quick brown fox jumps over the lazy dog.	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog.	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog.	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog.	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

Figure 3-11. Preparing a dataset for SkipGram

Word2Vec

- **Google News Word2Vec:**
 - **Corpus:** Trained on 100 billion words from Google News.
 - **Dimensions:** 300-dimensional vectors.
 - **Vocabulary Size:** 3 million words and phrases.
 - **Download link:** Available on Google Code Archive.
 - **Usage:** This is one of the most commonly used pre-trained embeddings in many NLP tasks due to its large corpus and robust performance.
- **Word2Vec on Wikipedia:**
 - **Corpus:** Trained on the Wikipedia corpus.
 - **Dimensions:** Typically 300-dimensional vectors.
 - **Vocabulary Size:** Depends on the version of Wikipedia used for training.
 - **Usage:** Similar to the Google News Word2Vec model but focused on Wikipedia articles, making it more domain-specific for encyclopedic content.
- **Twitter Word2Vec:**
 - **Corpus:** Trained on a large collection of tweets.
 - **Dimensions:** Typically 400-dimensional vectors.
 - **Vocabulary Size:** Several million words and phrases.
 - **Download link:** Available through specific research or community-contributed models.
 - **Usage:** Particularly useful for tasks involving social media text, where the vocabulary and language style is different from formal texts.
- **Word2Vec from Amazon Reviews:**
 - **Corpus:** Trained on Amazon product reviews, focusing on sentiment and product-related language.
 - **Dimensions:** Typically 100-300 dimensions.
 - **Vocabulary Size:** Several hundred thousand words.
 - **Usage:** Good for sentiment analysis and product-related NLP tasks due to its domain-specific nature.
- **Common Crawl Word2Vec:**
 - **Corpus:** Trained on Common Crawl data, which is a very large, open web corpus.
 - **Dimensions:** Typically 300-dimensional vectors.
 - **Vocabulary Size:** Millions of words and phrases.
 - **Usage:** Useful for tasks where general, web-based text data is required.

Word2Vec

- **Advantages:**
 - **Captures Semantic Relationships:** Learns word vectors that capture meanings and relationships between words.
 - **Efficient Training:** Uses neural networks for efficient training and representation learning.
- **Disadvantages:**
 - **Limited to Fixed Context Window:** Context window size is fixed, which may **not capture long-range dependencies**.
 - **No Handling of Out-of-Vocabulary (OOV) Words:** Cannot generate embeddings for words not seen during training.

GloVe (Global Vectors for Word Representation)

- **GloVe** is an unsupervised learning algorithm for obtaining vector representations for words by aggregating **global word-word co-occurrence statistics** from a corpus. Developed by **Stanford**, it focuses on **word co-occurrence matrices**.
- **How It Works:**
 - **Co-occurrence Matrix:** Constructs a matrix where entries represent how frequently word pairs occur together.
 - **Factorization:** Factorizes the matrix to obtain dense word vectors that capture semantic meaning. The goal is to approximate the original matrix using these lower-dimensional representations, making it easier to work with and revealing latent structures.

Common Pre-trained GloVe Embeddings

- **GloVe.6B**: Trained on a corpus of 6 billion tokens from Wikipedia and Gigaword.
- **GloVe.42B**: Trained on 42 billion tokens from a larger Common Crawl dataset.
- **GloVe.840B**: Trained on 840 billion tokens from an even larger Common Crawl dataset.
- **GloVe.Twitter**: Trained on tweets, capturing informal language and slang.

GloVe (Global Vectors for Word Representation)

- **Advantages:**
 - **Captures Global Statistics:** Uses **global word co-occurrence** information, leading to more **accurate** embeddings.
- **Disadvantages:**
 - **Fixed Representations:** Pre-trained embeddings may not capture domain-specific nuances.
 - **Requires Large Corpora:** Effective when trained on **large corpora**, which may not be available for all applications.
 - **Doesn't Handle OOV Words:** GloVe struggles with out-of-vocabulary (OOV) words, since the model only learns embeddings for words that appear in the training corpus.

FastText

- **FastText**, developed by **Facebook's** AI Research (FAIR), extends **Word2Vec** by representing words **as bags of character n-grams**. This approach allows FastText to generate embeddings for morphologically rich languages and **handle out-of-vocabulary (OOV)** words better.
- **How It Works:**
 - **Subword Information:** Words are represented as a **combination** of **character n-grams**, allowing the model to learn from subword information.
 - **Out-of-Vocabulary Words:** Can generate embeddings for OOV words by averaging the embeddings of its n-grams.
- **Advantages:**
 - **Handles Morphological Variations:** Captures subword information, which is useful for morphologically rich languages.
 - **OOV Handling:** Can generate embeddings for words not seen during training.
- **Disadvantages:**
 - **Increased Complexity:** Slightly more complex than Word2Vec due to subword representation.
 - **Potential Overfitting:** Can overfit on the n-grams if the corpus is small or noisy.

Choosing the Right Pre-trained Embeddings

- **General Tasks:** Google News Word2Vec, GloVe.
- **Social Media:** Twitter Word2Vec.
- **Sentiment Analysis:** Amazon Reviews Word2Vec.
- **Rare Words/Subwords:** FastText.

Summary

- **Word2Vec**, **GloVe**, and **FastText** are popular methods for learning word embeddings, each with its unique approach and strengths:
 - **Word2Vec**: Effective for capturing semantic relationships through local context with either CBOW or Skip-gram. However, it has limitations with out-of-vocabulary words and fixed context windows.
 - **GloVe**: Utilizes global word co-occurrence statistics to create embeddings, capturing broader semantic relationships. Pre-trained models are available but may not be adaptable to domain-specific needs.
 - **FastText**: Enhances Word2Vec by incorporating subword information, improving handling of out-of-vocabulary words and morphological variations.